Metodología de Sistemas II

TP Unidad 5 — Parte 2

Aseguramiento de la calidad del software aplicado a nuestra aplicación

Dato	Detalle
Equipo	ATM — Sistema de Gestión Integral "Educar para Transformar"
Integrante	Loza, Mateo
Comisión	1 — Grupo Tarde
Repositorio	github.com/lozamateo/centroeducativoATM
Aplicación	Web: https://centroeducativo-atm.vercel.app — App móvil: Expo (EAS Update)

1. Definición de calidad adoptada para el proyecto

Entre las definiciones del material (Pressman, Humphrey, Al Davis e IEEE) adoptamos la del glosario de la **IEEE**, porque une las dos perspectivas: el cumplimiento de los requerimientos y la satisfacción del usuario.

> *"El grado con el cual un sistema, componente o proceso cumple con los requerimientos y con las necesidades y expectativas del usuario."*

En nuestro sistema esto significa: (1) cumplir los requerimientos RF-01 a RF-22 y RNF-01 a RNF-05 del Plan de Trabajo, y (2) que las familias, docentes, alumnos, enfermería y la dirección puedan usarlo sin errores y con confianza. Como dice Tom De Marco, "no se puede controlar lo que no se puede medir": por eso cada atributo de calidad lleva una evidencia o una métrica.

2. Atributos de calidad según el modelo FURPS+

Atributo	Cómo se cumple en nuestra aplicación	Evidencia / métrica
F — Funcionalidad (incluye seguridad)	Matrícula, deportes, transporte, comedor, ficha médica con credencial QR, portales por rol y reportes MIS. Seguridad: contraseñas cifradas con bcrypt, token JWT con vencimiento de 8 h, control de acceso por roles (RBAC) y Row Level Security en la base: un tutor solo puede ver a sus propios hijos.	Pruebas de API durante el desarrollo: un tutor que pide un alumno ajeno recibe 403; sin token, 401. Pruebas E2E de login (Cypress).
U — Facilidad de uso	La web y la app tienen las mismas pantallas, colores y textos (código compartido en *shared/*). Diseño adaptable a celular, mensajes de error claros en español, confirmación antes de cada baja y botones de usuarios de demostración. La documentación está en el README.	Recorrido completo de cada rol en navegador de escritorio y de celular (390 px) sin desbordes de pantalla. La inscripción a un deporte se hace en 3 pasos (alumno → Deportes → Inscribir).
R — Fiabilidad	Las reglas de negocio se validan en dos capas: en la API (mensajes claros) y en la base de datos (triggers y restricciones únicas, con transacciones ACID). Si una regla falla, la operación se revierte completa (rollback). Cada despliegue en Vercel queda guardado y se puede volver al anterior.	Las reglas de inscripción tienen 100 % de cobertura de pruebas. Tasa de error observada en Vercel (ver no conformidad NC-03).
P — Rendimiento	API serverless en Vercel con pool de conexiones a PostgreSQL (Supabase). Los reportes usan vistas SQL ya preparadas (padrón por curso, padrón de transporte, ficha 360°).	Objetivo del Plan de Trabajo: reportes en menos de 5 s. Se mide en la pestaña Observabilidad de Vercel. Build de la web: unos 8 s.

S — Soporte	Arquitectura en capas (rutas → seguridad → servicios → persistencia), TypeScript, ESLint, configuración por variables de entorno y textos en es-AR. El código común entre web y app evita duplicar cambios.	ESLint sin errores. 93,5 % de cobertura de líneas sobre los módulos probados. README con instalación y despliegue.
+ Implementación	React 19 + Vite, React Native + Expo SDK 58, Node.js 22 + Express 5, PostgreSQL.	package.json y app.json versionados.
+ Interfaz	API REST con JSON y encabezado *Authorization: Bearer <JWT>*. Integración con Supabase, Vercel, GitHub y Expo.	Endpoints documentados en el README.
+ Operaciones	Despliegue automático en cada merge a main. Gestión de usuarios y roles desde el panel. Auditoría de cada alta, modificación y baja.	Tabla *auditoria* con usuario y fecha.
+ Empaquetamiento	Web publicada en Vercel. App distribuida con Expo Go y EAS Update; se puede generar un instalable con EAS Build.	URL pública y actualización publicada en Expo.
+ Legales	Se manejan datos sensibles de salud de menores (Ley 25.326 de Protección de Datos Personales). La credencial QR pública no muestra datos médicos: solo si el alumno está habilitado. Las dependencias usan licencias de código abierto.	Endpoint público de credencial con datos mínimos.

3. Costo de la calidad

Costo de la Calidad = Costo de Conformidad (prevención + evaluación) + Costo de No Conformidad (fallos internos + externos)

Categoría	Actividades en nuestro proyecto
Prevención	Plan de Trabajo con requerimientos y reglas de negocio definidos antes de programar; reglas implementadas también en la base de datos; estándares de codificación con ESLint; pre-commit hook (Husky) que bloquea commits con errores; plantilla de Pull Request con checklist.
Evaluación	21 pruebas unitarias (Vitest), 7 pruebas E2E (Cypress), pipeline de CI en cada push y en cada PR, recorrido manual de cada rol en web y app, y revisión visual de pantallas.
Fallos internos	Defectos encontrados antes de llegar al usuario: textos sin tildes, logo en blanco en el panel y una materia sin docente asignado (NC-01, NC-02 y NC-04).
Fallos externos	Defecto encontrado ya en producción: el login devolvía "Error interno del servidor" (NC-03). Se resolvió antes de la presentación, pero costó más tiempo de diagnóstico que los fallos internos.

Esto coincide con la Tabla 6 del material (Pressman): corregir un error en requisitos cuesta 1, y en producción entre 40 y 1000 veces más. Por eso las reglas de inscripción (máximo 2 deportes, sin superposición horaria, apto físico vigente) se definieron en el Plan de Trabajo y se implementaron desde el diseño de la base. Así, un error de lógica se detecta en las pruebas y no cuando una familia ya inscribió mal a su hijo.

4. SQA y SQC en nuestro proyecto

Control de calidad (SQC) — revisa el producto	Aseguramiento de la calidad (SQA) — revisa el proceso
Pruebas unitarias de las reglas de negocio y de las validaciones (Vitest).	Política de Pull Requests: todo cambio entra por PR con checklist (.github/pull_request_template.md).

Pruebas E2E del login y de la inscripción a deportes (Cypress).	Pipeline de CI obligatorio antes del merge y protección de la rama main (Issue #5).
Análisis estático con ESLint (y SonarCloud cuando se active, Issue #6).	Pre-commit hook (Husky): el proceso impide registrar código que no pasa lint y pruebas.
Prueba manual de cada rol en la web y en la app antes de la presentación.	Registro de tareas en Issues y tablero Kanban; trazabilidad Issue → PR → pipeline → despliegue.
Verificación de los datos cargados (por ejemplo, materias sin docente).	Auditoría de proceso con plantilla de chequeo y registro de no conformidades (punto 6).

Como explica el material, hacer solo pruebas no asegura la calidad: verifica el producto, pero no garantiza que el proceso se respete. Por eso combinamos las dos cosas: el SQC detecta los defectos y el SQA controla que se sigan los procedimientos que evitan que vuelvan a aparecer.

5. Funciones del SQA aplicadas

Rol del SQA	Cómo lo aplicamos
Policía del proceso	El pipeline y el pre-commit hacen cumplir automáticamente el proceso definido: sin lint, pruebas y build en verde no se integra. La auditoría del punto 6 juzga el proceso, no el producto.
Abogado del cliente	Cada regla de negocio se traduce en un mensaje claro para la familia (por ejemplo "Apto físico vencido o ausente"). Probamos la app "como cliente" con los usuarios de demostración de cada rol.
Analista	Se recolectan datos del proceso y del producto: cobertura, resultados del pipeline, tasa de error de Vercel y registro de auditoría.
Proveedor de información	La página "Calidad y CI/CD" de la web y el Tablero de gestión muestran métricas objetivas para que la dirección tome decisiones.
Responsable del proceso	Definimos la política de PR, el checklist, las etiquetas de Issues y el flujo Kanban desde las primeras etapas del proyecto.

Consideración sobre la independencia (punto 3.3.5 del material): al ser un equipo unipersonal, la misma persona desarrolla y controla, lo que reduce la objetividad. Lo compensamos con controles automáticos que no dependen del autor (CI, pre-commit, pruebas), con un checklist fijo de autorrevisión y con la revisión externa del docente en las tutorías, que cumple el papel de canal independiente para escalar no conformidades.

6. Aseguramiento de la calidad de procesos y productos (PPQA — CMMI nivel 2)

a) Metas de PPQA y cómo las cumplimos

Meta	Aplicación
Evaluar objetivamente procesos y productos contra lo definido.	Plantilla de chequeo (punto b), completada al cierre del sprint de presentación.
Identificar y documentar no conformidades.	Registro de no conformidades (punto c), también cargadas como Issues.
Informar los resultados a quienes usan el proceso y a la gerencia.	Este informe, la página "Calidad y CI/CD" y las tutorías con el docente.
Asegurar que las no conformidades se traten.	Cada NC tiene acción correctiva, responsable y estado; se cierra al hacer merge del PR que la resuelve.

b) Plantilla de chequeo — Auditoría del proceso (27/10/2026)

N°	Criterio del proceso	Resultado	Evidencia
1	Todo cambio a main entra por Pull Request.	**Parcial**	PR #1 y PR #8 sí. Los primeros commits del proyecto entraron por push directo.
2	El pipeline de CI pasa antes del merge.	**Cumple**	PR #8: lint, pruebas, build y E2E en GitHub Actions.
3	Las reglas de negocio tienen pruebas automatizadas.	**Cumple**	100 % de cobertura en server/rules/deportes.js.
4	Las tareas se registran como Issues con etiquetas.	**Cumple**	Issues #2 a #7 con etiquetas; #2 a #5 cerradas.
5	Los secretos no se guardan en el repositorio.	**Cumple**	.env en .gitignore; variables cargadas en Vercel.
6	Los secretos no se comparten por canales inseguros.	**No cumple**	Ver NC-05.
7	La documentación está actualizada.	**Parcial**	README actualizado; la Wiki está pendiente (Issue #7).
8	El análisis estático corre en el pipeline.	**Parcial**	ESLint sí; SonarCloud pendiente de activar (Issue #6).
9	Se verifica la configuración de producción después de cada despliegue.	**No cumple**	Ver NC-03: se detectó recién al usar el login.

c) Registro de no conformidades

N°	No conformidad	Detectada en	Acción correctiva	Estado
NC-01	Textos de la interfaz y mensajes sin tildes ni eñes.	Revisión visual (interno)	Corrección de textos en web, app y API; script SQL 002_tildes.sql para la base ya creada.	**Cerrada**
NC-02	El logo quedaba en blanco en el panel si la imagen no cargaba.	Pruebas en navegador (interno)	Logo con imagen de respaldo (isotipo y nombre) cuando falla la carga.	**Cerrada**
NC-03	El login en producción respondía "Error interno del servidor".	Producción (externo)	Causa: faltaba la variable JWT_SECRET en Vercel. Se agregó y se redesplegó. Prevención: checklist de variables de entorno en la Wiki (Issue #7).	**Cerrada**
NC-04	La materia "Juego y expresión" quedaba sin docente asignado.	Verificación de datos (interno)	Se corrigió el nombre en los datos y se reasignó el docente con el script 003_datos_demo.sql.	**Cerrada**
NC-05	Credenciales (contraseña de la base y token de Expo) compartidas en un canal de comunicación no seguro.	Auditoría de proceso	Rotar la contraseña de Supabase y revocar el token de Expo; no volver a compartir secretos fuera del gestor de variables.	**Pendiente (después de la presentación)**
NC-06	El logo depende de una imagen alojada en una cuenta externa de un tercero.	Auditoría de proceso	Guardar el logo dentro del repositorio.	**Pendiente**

7. Métricas de calidad

Métrica	Valor	Fuente
Pruebas unitarias	21 (100 % pasan)	npm run test:coverage (Vitest)
Pruebas E2E	7	cypress/e2e (login e inscripción a deportes)
Cobertura de líneas / ramas / funciones	93,5 % / 84,9 % / 80 %	Reporte de cobertura de Vitest
Cobertura de las reglas de inscripción	100 %	server/rules/deportes.js
Errores de lint	0	npm run lint (ESLint)
No conformidades detectadas / cerradas	6 / 4	Registro del punto 6.c
Defectos internos vs. externos	3 internos / 1 externo	NC-01, 02 y 04 / NC-03
Tasa de error de la API en producción	5,3 % (antes de corregir NC-03)	Vercel → Observabilidad
Issues abiertas / cerradas	2 / 4	GitHub Issues

8. Conclusión

La calidad de nuestro sistema no depende solo de probar al final, sino de un proceso que la sostiene en todo el ciclo de vida, como plantea el material: requerimientos y reglas definidos en el Plan de Trabajo (prevención), pruebas automatizadas y manuales (evaluación), y un proceso controlado con CI, Pull Requests, Issues y auditorías (SQA/PPQA). El registro de no conformidades confirmó la idea del costo de la calidad: los defectos internos se resolvieron rápido, mientras que el único defecto externo (NC-03) fue el más costoso de diagnosticar. Como próximos pasos: cerrar NC-05 y NC-06, activar SonarCloud, documentar la Wiki de despliegue y agregar una verificación automática de la configuración de producción después de cada despliegue.

Anexo — Acceso a la aplicación

Acceso	Detalle
Web	https://centroeducativo-atm.vercel.app
App móvil	Expo Go → exp://u.expo.dev/92e57b9c-11c3-415e-9537-b4e6e9713139/group/1c849c1b-5c70-423b-bfec-721103877306

Rol	Usuario (email)	Contraseña
Administrador	admin@educar.com	admin123
Familia (tutor)	tutor@educar.com	tutor123
Familia 2	mariana@educar.com	familia123
Docente	docente@educar.com	docente123
Alumno	alumno@educar.com	alumno123
Enfermería	enfermeria@educar.com	enfermeria123

Bibliografía: material de la cátedra "Capítulo 3 — Aseguramiento de la calidad del software" (Pressman, Humphrey, Al Davis, IEEE, Galin, SEI, modelo FURPS+ de Grady y Caswell, CMMI-PPQA).